

Week4_예습과제

통계 수정 삭제

카사바 · 방금 전

❤️ 0

Euron_2026-1



▼ 목록 보기

3/3



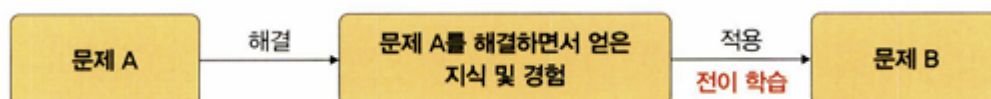
5장 합성곱 신경망 I

5.3 전이 학습

합성곱 신경망 기반의 딥러닝 모델을 제대로 훈련시키려면 많은 양의 데이터가 필요하지만 충분히 큰 데이터셋을 얻는 것이 쉽지 않다. 큰 데이터셋을 확보하려면 시간과 돈이 많이 필요하기 때문이다. 이런 현실적인 어려움을 해결한 것이 전이 학습(transfer learning)이다.

전이 학습: 아주 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것

사전 훈련된 모델(네트워크): 아주 큰 데이터셋을 사용하여 훈련된 모델



전이 학습을 위한 방법으로는 특성 추출과 미세 조정 기법이 있다.

5.3.1 특성 추출 기법

특성 추출(feature extractor)은 ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막 완전연결층 부분만 새로 만든다. 즉, 학습할 때는 마지막 완전연결층(이미지의 카테고리를 결정하는 부분)만 학습하고 나머지 계층들은 학습되지 않도록 한다.

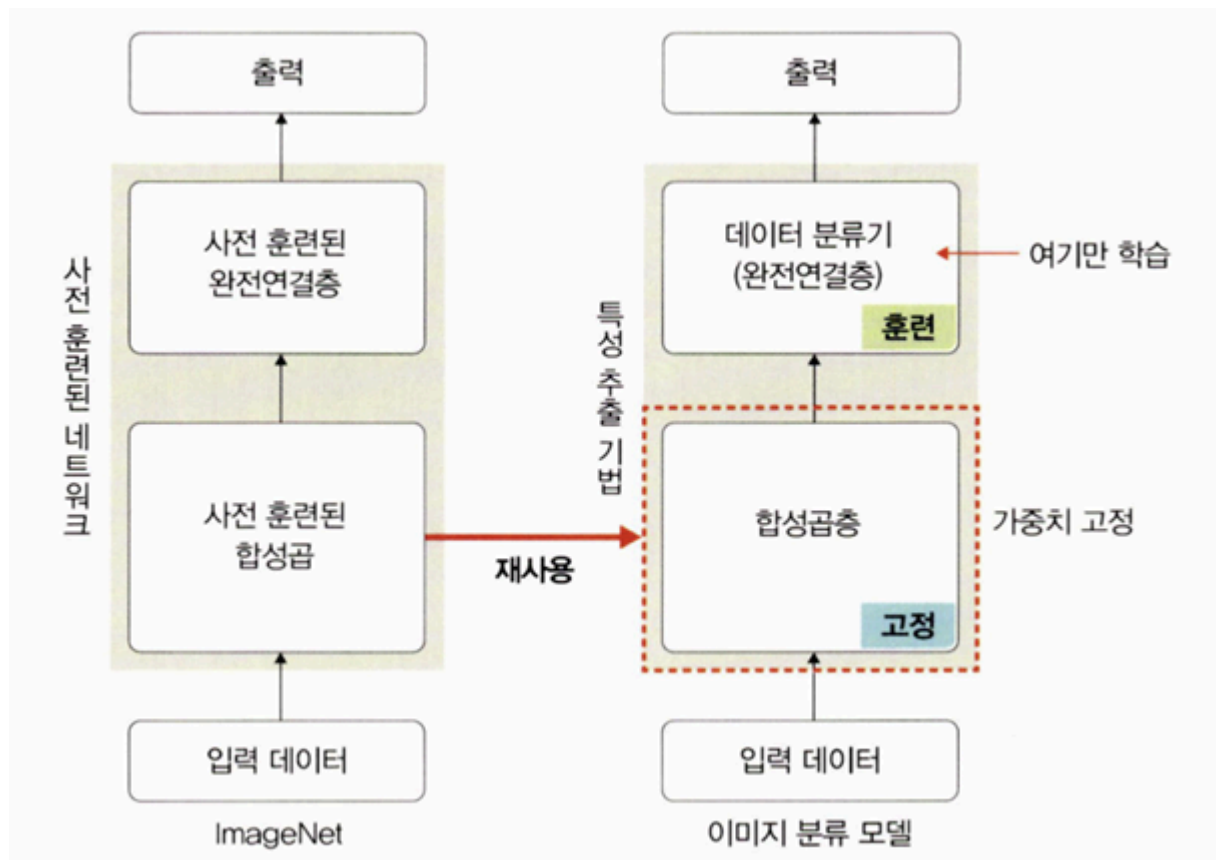
특성 추출은 이미지 분류를 위해 두 부분으로 구성된다.

- 합성곱층: 합성곱층과 풀링층으로 구성
- 데이터 분류기(완전연결층): 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류하는 부분

사전 훈련된 네트워크의 합성곱층(가중치 고정)에 새로운 데이터를 통과시키고, 그 출력을 데이터 분류기에서 훈련시킨다.

사용 가능한 이미지 분류 모델은 다음과 같다:

- Xception
- Inception V3
- ResNet50
- VGG16
- VGG19
- MobileNet



```

data_path = '/content/drive/MyDrive/train' # 이미지 데이터가 위치한 경로 지정

transform = transforms.Compose(
    [
        transforms.Resize([256, 256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor()
    ] ①
)
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
) ②
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
) ③

print(len(train_dataset))

```

① torchvision.transform은 이미지 데이터를 변환하여 모델(네트워크)의 입력으로 사용할 수 있도록 해준다.

```

transform = transforms.Compose([transforms.Resize([256, 256]),
                                ①
                                transforms.RandomResizedCrop(224), transforms.RandomHorizontalFlip(),
                                ②
                                transforms.ToTensor())])
                                ③
                                ④

```

① Resize: 이미지의 크기를 256 x 256 크기로 조정한다.

② RandomResizedCrop: 이미지를 랜덤한 크기 및 비율로 자른다. Resize는 합성곱층을 통과하기 위해 이미지 크기를 조정하는 전처리 과정이라면, RandomResizedCrop은 데이터 확장 용도로 사용된다.

③ RandomHorizontalFlip: 이미지를 랜덤하게 수평으로 뒤집는다.

④ ToTensor: 이미지 데이터를 텐서로 변환한다.

② datasets.ImageFolder는 데이터로더가 데이터를 불러올 대상(혹은 경로)과 방법(혹은 전처리)을 정의한다.

```
train_dataset = torchvision.datasets.ImageFolder(data_path,
                                                    (a)
                                                    transform=transform)
                                                    (b)
```

- ① 첫 번째 파라미터: 불러올 데이터가 위치한 경로
- ② transform: 이미지 데이터에 대한 전처리

③ 데이터로더는 데이터를 불러오는 부분으로 ImageFolder(train_dataset)을 할당하는데, 이때 한 번에 불러올 데이터양을 결정하는 batch_size를 지정한다. 그리고 추가적으로 데이터를 무작위로 섞을 것인지도 설정한다.

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32,
                                              (a)                (b)
                                              num_workers=8, shuffle=True)
                                              (c)                (d)
```

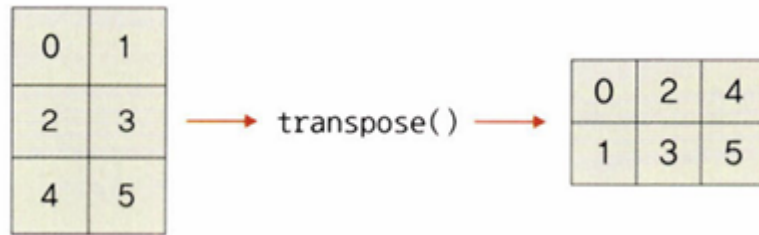
- ① 첫 번째 파라미터: 데이터셋 지정
- ② batch_size: 한 번에 불러올 데이터양을 결정하는 배치 크기 설정
- ③ num_workers: 데이터를 불러올 때 하위 프로세스를 몇 개 사용할지 설정한다. 이때 너무 많은 하위 프로세스를 설정하게 되면 오류가 발생하거나 메모리 부족 현상이 발생할 수 있다.
- ④ shuffle: 데이터를 무작위로 섞을 건지를 지정한다. shuffle=True로 설정하면 데이터를 무작위로 섞는다.

코드 5-14 학습에 사용될 이미지 출력

```
samples, labels = iter(train_loader).next() ----- ①
classes = {0:'cat', 1:'dog'} ----- 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): ----- 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) ----- 레이블 정보(클래스)를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0))) ----- ②
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

① 반복자(iterator, for 구문과 같은 효과)를 사용하려면 iter()와 next()가 필요하다. iter()는 전달된 데이터의 반복자를 꺼내 반환하며, next()는 그 반복자가 다음에 출력해야 할 요소를 반환한다. 앞의 코드에서 반복자는 train_loader가 되기 때문에 train_loader에서 samples와 labels를 순차적으로 꺼내서 저장한다.

② np.transpose는 행과 열을 바꿈으로써 행렬의 차원을 바꿔 준다.



행렬의 차원을 변경하는 이유는 행렬의 내적 연산 때문이다. 두 행렬의 행과 열의 수가 달라 내적이 불가능할 때는 np.transpose()나 np.reshape() 등으로 차원을 조정해야 한다.

곱셈, 덧셈

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

A B C

$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

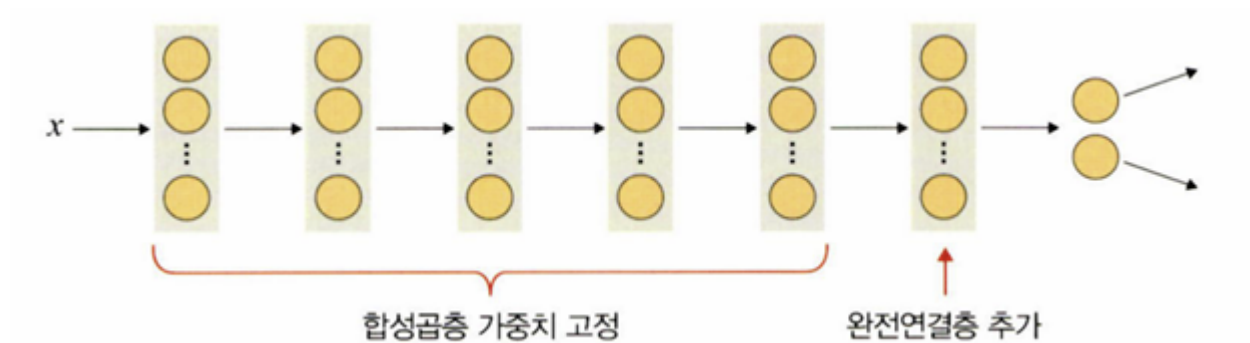
ResNet18: 50개의 계층으로 구성된 합성곱 신경망이다. 하지만 입력 제약이 매우 크고, 충분한 메모리(RAM)가 없으면 학습 속도가 느릴 수 있다는 단점이 있다.

```
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False ..... ①

set_parameter_requires_grad(resnet18)
```

① 역전파 중 파라미터들에 대한 변화를 계산할 필요가 없음을 나타낸다. 모델 일부(합성곱층과 풀링층)를 고정하고 나머지를 학습한다.

내려받은 ResNet18의 마지막 부분에 완전연결층을 추가한다. 이 완전연결층은 개와 고양이 클래스를 분류하는 용도로 사용된다.



```

def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('/content/drive/MyDrive/data/catanddog/' + '*.pth') ①
    saved_models.sort() # 불러온 .pth 파일들을 정렬
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders: # 테스트 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad(): # autograd를 사용하지 않겠다는 의미
                outputs = model(inputs) # 데이터를 모델에 적용한 결과를 outputs에 저장

            _, preds = torch.max(outputs.data, 1) ②
            preds[preds>=0.5] = 1 # torch.max로 출력된 값이 0.5보다 크면 올바르게 예측
            preds[preds<0.5] = 0 # torch.max로 출력된 값이 0.5보다 작으면 틀리게 예측
            running_corrects += preds.eq(labels.cpu()).int().sum() ③

        epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 테스트 데이터의 정확도 계산
        print('Acc: {:.4f}'.format(epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc
            acc_history.append(epoch_acc.item())
            print()

        time_elapsed = time.time() - since
        print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60,
                                                                time_elapsed % 60))

        print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history # 계산된 정확도 반환

```

① glob은 현재 디렉터리에서 원하는 파일들만 추출하여 가져올 때 사용한다. 즉, 해당 경로에서 pth 확장자를 갖는 파일을 가져오라는 의미이다. pth라는 확장자를 갖는 파일은 훈련 데이터로 모델을 훈련시킬 때 생성된 파일이다.

② torch.max는 주어진 텐서 배열의 최댓값이 들어 있는 index를 반환하는 함수이다.

③ preds.eq(labels)는 preds 배열과 labels가 일치하는지 검사하는 용도로 사용한다. 또한, 뒤에 사용된 .sum()은 모델의 예측 결과와 정답(레이블)이 일치하는 것들의 개수 합을 숫자로

출력한다.

코드 5-28 예측 이미지 출력을 위한 전처리 함수

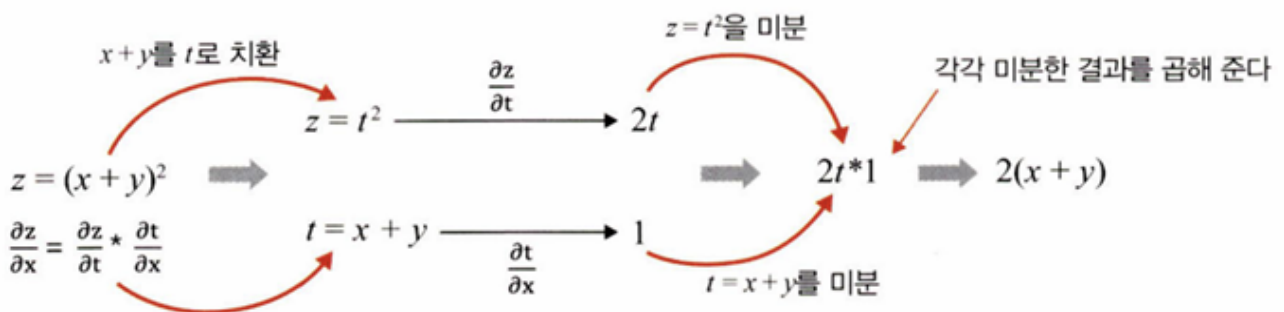
```
def im_convert(tensor):  
    image = tensor.clone().detach().numpy() ----- ①  
    image = image.transpose(1, 2, 0)  
    image = image * (np.array((0.5,0.5,0.5)) + np.array((0.5,0.5,0.5)))  
    image = image.clip(0, 1) ----- ②  
    return image
```

① `tensor.clone()`은 기존 텐서의 내용을 복사한 텐서를 생성하겠다는 의미이며, `detach()`는 기존 텐서에서 기울기가 전파되지 않는 텐서이다. 즉 기존 텐서를 복사한 새로운 텐서를 생성하지만 기울기에 영향을 주지는 않겠다는 의미이다.

② `clip()`은 입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용한다. `image.clip(0,1)`은 `image` 데이터를 0과 1 사이의 값으로 제한하겠다는 의미이다.

<연쇄 법칙>

체인룰(chain rule): 합성 함수의 미분법



코드 5-29 개와 고양이 예측 결과 출력

```
classes = {0:'cat', 1:'dog'} ----- 개와 고양이 두 개에 대한 레이블

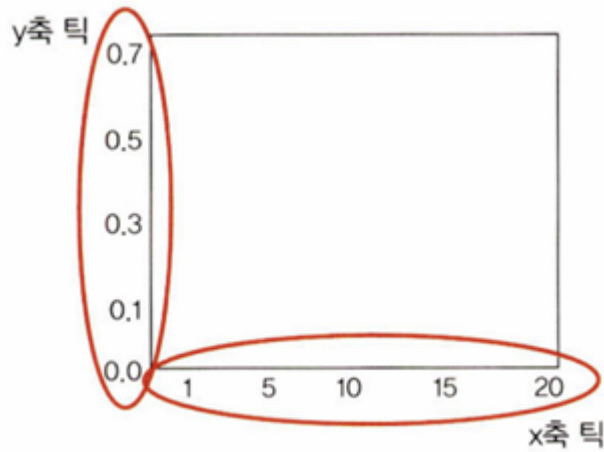
dataiter = iter(test_loader) ----- 테스트 데이터셋을 가져옵니다.
images, labels = dataiter.next() ----- 테스트 데이터셋에서 이미지와 레이블을 분리하여 가져옵니다.
output = model(images)
_, preds = torch.max(output, 1)

fig = plt.figure(figsize=(25,4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[]) ----- ①
    plt.imshow(im_convert(images[idx])) ----- 이미지 출력을 위해 코드 5-28에서 정의한 im_convert 함수를 적용
    a.set_title(classes[labels[i].item()])
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]), str(classes[labels[idx].
        item()]))), color=("green" if preds[idx]==labels[idx] else "red")) ----- ②
plt.show()
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0) ----- ③
```

① add_subplot은 한 화면에 여러 개의 이미지를 담기 위해 사용한다. 이때 사용되는 파라미터는 다음과 같다.

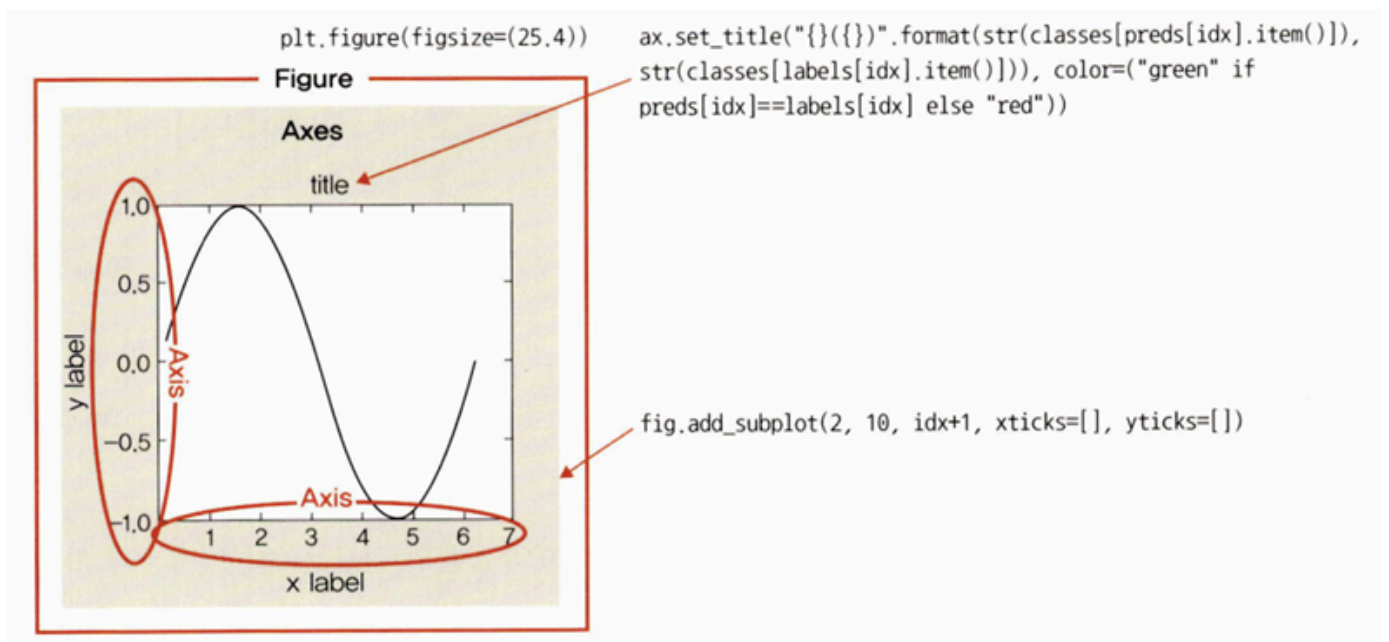
`add_subplot(2, 10, idx+1, xticks=[], yticks=[])`
 ① ② ③ ④

- ① 첫 번째 파라미터: 행의 수를 의미한다. 이 경우 이미지를 두 줄로 출력한다.
- ② 두 번째 파라미터: 열의 수를 의미한다. 이 경우 한 줄에 열 개의 이미지를 출력한다.
- ③ 세 번째 파라미터: 인덱스를 의미한다. 행과 열을 기준으로 순차적으로 이미지를 출력한다.
- ④ 네 번째 파라미터: 틱(tick)을 삭제하겠다는 의미.



② `classes[preds[idx].item()]`은 `preds[idx].item()` 값이 `classes`로 정의된 '0'과 '1' 중 어떤 값을 갖는지 판별하겠다는 의미이다. 즉 이 값이 0이면 고양이, 1이면 개로 출력된다.

③ Figure 안에서 서브플롯(다음 그림에서 Axes에 해당)의 위치를 조정할 때 사용한다. left, bottom, right, top으로 이미지 위치를 조정한다. 그리고 `hspace`와 `wspace`를 사용해 서브플롯 간의 간격(너비와 높이의 비율)을 조정할 수 있다.



5.3.2 미세 조정 기법

미세 조정 기법(fine-tuning): 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식. 즉, 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것.

미세 조정 과정에서 많은 연산량이 필요하기 때문에 CPU보다는 GPU를 사용하는 것이 낫다.

미세 조정 기법은 훈련시키려는 데이터셋의 크기와 사전 훈련된 모델에 따라 다음과 같이 전략을 세울 수 있다.

- 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우: 모델 전체를 재학습시킨다.
- 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우: 합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기(완전연결층)를 학습시킨다. 전체를 학습시키는 것보다는 강한 특징이 나타나는 합성곱층 뒷부분과 데이터 분류기만 새로 학습시켜도 최적의 성능을 낼 수 있다.
- 데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우: 합성곱층 일부분과 데이터 분류기를 학습시킨다. 데이터가 적기 때문에 일부 계층에 미세 조정 기법을 적용한다고 해도 효과가 없을 수 있다. 따라서 합성곱층 중 어디까지를 새로 학습시켜야 할지를 적당히 설정해 주어야 한다.
- 데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우: 데이터 분류기만 학습시킨다. 데이터가 적기 때문에 많은 계층에 미세 조정 기법을 적용하면 과적합이 발생할 수 있다.

미세 조정은 파라미터 업데이트 과정에서 파라미터에 큰 변화를 주면 과적합 문제가 발생할 수 있기 때문에 정교하고 미세한 파라미터 업데이트가 필요하다.

5.4 설명 가능한 CNN

설명 가능한 CNN(explainable CNN): 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술

CNN은 내부에서 어떻게 동작하는지 설명하기 어렵다. 따라서 CNN으로 얻은 결과는 신뢰하기 어렵다. 이를 해결하려면 CNN 처리 과정을 시각화해야 한다.

CNN의 시각화 방법에는 필터에 대한 시각화와 특성 맵에 대한 시각화가 있지만, 여기서는 특성 맵에 대한 시각화만 다룬다.

5.4.1 특성 맵 시각화

특성 맵(feature map)(혹은 활성화 맵): 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과. 따라서 특정 입력 이미지에 대한 특성 맵을 시각화한다는 의미는 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 돕는 것이다.

PIL(Python Image Library): 다양한 이미지 파일 형식을 지원하며, 강력한 이미지 처리와 그래픽 기능을 제공하는 이미지 프로세싱 라이브러리

```
def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x) # ①
```

① 로그 소프트맥스(log_softmax(x)): 신경망 말단의 결괏값들을 확률 개념으로 해석하기 위해 소프트맥스 함수의 결과에 log 값을 취한 연산. 소프트맥스 대신 로그 소프트맥스를 쓰는 이유는 소프트맥스는 기울기 소멸 문제에 취약하기 때문이다. 다음은 소프트맥스와 로그 소프트맥스에 대한 수식이다.

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\begin{aligned} \text{logsoftmax} &= \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right) \\ &= x_i - \log\left(\sum_{j=1} e^{x_j}\right) \end{aligned}$$

특성 맵은 합성곱층에서 입력 이미지와 필터를 연산하여 얻은 결과이다. 따라서 합성곱층에서 입력과 출력을 알 수 있다면 특성 맵에 대한 값들을 확인할 수 있다.

특성 맵을 확인하기 위한 클래스 정의

```
class LayerActivations:
    features = []
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn) # ①

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self): # hook 삭제
        self.hook.remove()
```

① 파이토치는 매 계층마다 print 문을 사용하지 않더라도 hook 기능을 사용하여 각 계층의 활성화 함수 및 기울기 값을 확인할 수 있다. 따라서 register_forward_hook의 목적은 순전파 중에 각 네트워크 모듈의 입력 및 출력을 가져오는 것이다. hook을 사용하면 중간 결괏값들을 확인할 수 있다.

```
img = cv2.imread('/content/drive/MyDrive/cat.jpg')
plt.imshow(img)
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR) # ①
img = ToTensor()(img).unsqueeze(0) # ②
print(img.shape)
```

① cv2.resize는 이미지 크기를 변경할 때 사용한다.

```
cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)
```

(a) (b) (c)

- ㉠ 첫 번째 파라미터: 변경할 이미지 파일
- ㉡ 두 번째 파라미터: 변경될 이미지 크기를 (너비, 높이)로 지정
- ㉢ interpolation: 보간법

이미지 크기를 변경할 경우 변형된 이미지의 픽셀을 추정해서 값을 할당해야 한다. 이미지 비율을 변경하면 존재하지 않는 영역에 새로운 픽셀 값을 매핑하거나 존재하는 픽셀들을 압축해서 새로운 값을 할당해야 한다. 이러한 상황을 피하고자 이미지상에 존재하는 픽셀 데이터들에 대해 근사 함수를 적용해서 새로운 픽셀 값을 구하는 것이 보간법이다. 즉, 추정해야 하는 픽셀을 보간법을 이용하여 값을 할당한다.

② `ToTensor()(img).unsqueeze(0)`에서 사용된 `unsqueeze()`는 1차원 데이터를 생성하는 함수이다. 즉, 이미지 데이터를 텐서로 변환하고, 그 변환된 데이터를 1차원으로 변경하겠다는 의미이다.

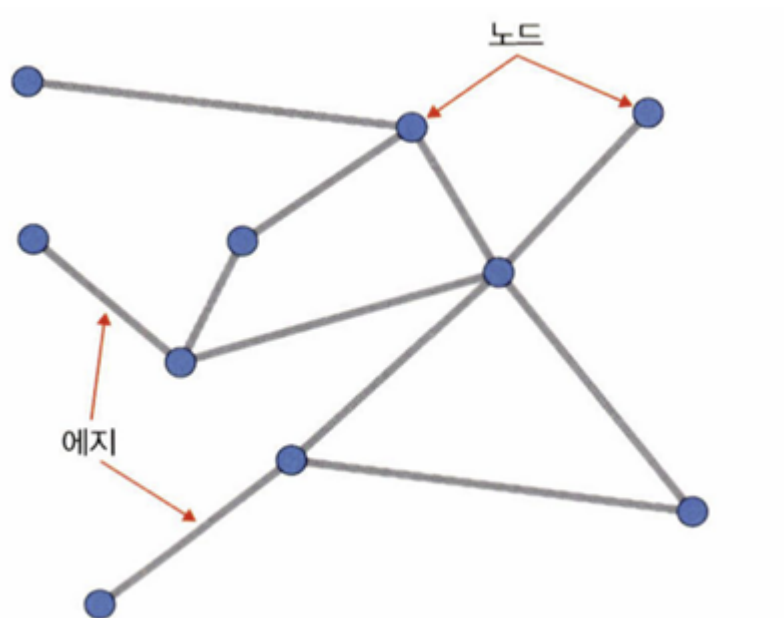
출력층에 가까울수록 원래 형태를 찾아볼 수 없고, 이미지 특징들만 전달되는 것을 확인할 수 있다.

5.5 그래프 합성곱 네트워크

그래프 합성곱 네트워크(graph convolutional network)는 그래프 데이터를 위한 신경망이다.

5.5.1 그래프란

그래프: 방향성이 있거나 없는 에지로 구성된 노드의 집합



노드: 파란 원

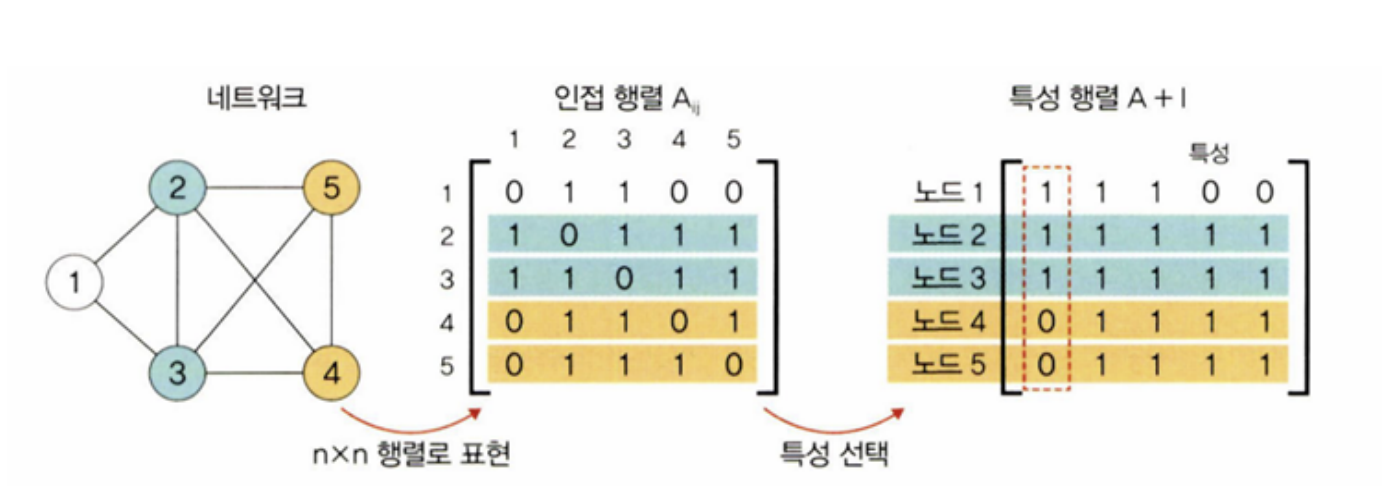
에지: 두 노드를 연결한 선

노드는 원소들을 의미하고, 에지는 결합 방법(single, double, triple, aromatic 등)을 의미한다.

5.5.2 그래프 신경망

그래프 신경망(Graph Neural Network, GNN): 그래프 구조에서 사용하는 신경망

그래프 데이터에 대한 표현은 다음과 같이 두 단계로 이루어진다.



1단계. 인접 행렬(adjacency matrix)

- 위 그림 왼쪽과 같은 네트워크가 있을 때 노드 n 개를 $n \times n$ 행렬로 표현한다.
- 인접 행렬 내의 값은 'i와 j의 관련성 여부(A_{ij})'를 만족하는 값으로 채운다.

즉, 인접 행렬 과정은 컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정이다.

2단계. 특성 행렬(feature matrix)

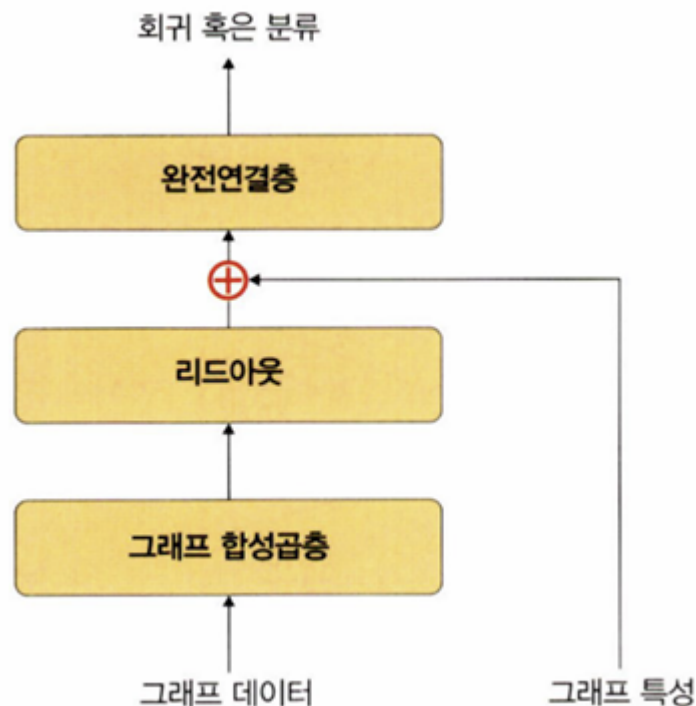
- 인접 행렬만으로는 특성을 파악하기 어렵기 때문에 단위 행렬을 적용한다.
- 각 입력 데이터에서 이용할 특성을 선택한다.
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미한다.

즉, 특성 행렬 과정을 거쳐 그래프 특성(graph feature)이 추출된다.

5.5.3 그래프 합성곱 네트워크

그래프 합성곱 네트워크(Graph Convolutional Network, GCN): 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

그래프 합성곱 네트워크 구조는 다음과 같다.

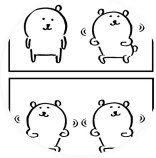


리드아웃(readout)은 특성 행렬을 하나의 벡터로 변환하는 함수이다. 즉, 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성한다.

GCN에서 가장 중요한 부분은 그래프 합성곱층이다. 이를 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환되어 딥러닝 알고리즘을 적용할 수 있기 때문이다. 기존 그래프 형태의 데이터로는 딥러닝 알고리즘을 적용할 수 없다.

GCN은 다음과 같은 곳에서 활용된다:

- SNS에서 관계 네트워크
- 학술 연구에서 인용 네트워크
- 3D Mesh



카사바

안녕하세요 :)



이전 포스트

Week2_예습과제

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate